Entrega 1: Perceptrón Multicapa para clasificación de texto

Clasificación de 20 Newsgroups con Keras/TensorFlow

Nicolás Garcés (código 218066) **Juan Pablo Ante** (código 2140132)

Escuela de Ingeniería de Sistemas y Computación
Universidad del Valle

2026-2

Resumen

Este trabajo presenta la primera entrega del proyecto de Redes Neuronales: la construcción, el entrenamiento y la evaluación de un Perceptrón Multicapa (MLP) para clasificar documentos del corpus *20 Newsgroups* en 20 categorías. La entrada son textos crudos procesados por una capa `TextVectorization` que aprende un vocabulario de 20,000 tokens y calcula representaciones TF-IDF; sobre ellas se apila una capa densa de 256 neuronas con activación ReLU y dropout, y una capa de salida softmax de 20 unidades. El modelo se entrena con Adam y entropía cruzada categórica, con parada temprana guiada por la exactitud de validación. En el conjunto de prueba, nunca usado durante el entrenamiento, se obtiene una exactitud de 0.6771, un F1 macro de 0.6663, un F1 ponderado de 0.6762 y una exactitud top-3 de 0.8452. Se comparan 9 configuraciones que varían profundidad, dropout, tamaño de vocabulario, balanceo de clases, tasa de aprendizaje y regularización L2. Los resultados muestran que una representación TF-IDF combinada con una MLP supera claramente al azar y mejora a un modelo lineal, pero que el sobreajuste y el solapamiento semántico entre clases limitan el desempeño. Se identifican las confusiones entre grupos de computación, religión y política, y se discuten las limitaciones de las representaciones *bag-of-words* como motivación para la segunda entrega.

Palabras clave: clasificación de texto, Perceptrón Multicapa, TF-IDF, 20 Newsgroups, Keras, TensorFlow.

Índice

1. Introducción **3**

2. Marco conceptual **3**

2.1. Representación TF-IDF 3

2.2. Perceptrón Multicapa 4

3. Conjunto de datos **4**

3.1. Descripción 4

3.2. Preprocesamiento 5
3.3. Análisis exploratorio 6

4. Arquitectura de la red **7**
4.1. Decisiones de diseño 7

5. Entrenamiento y ajuste de parámetros **7**
5.1. Protocolo 8
5.2. Espacio de búsqueda 8
5.3. Costo computacional 9

6. Resultados y análisis **9**
6.1. Desempeño del modelo principal 9
6.2. Análisis por clase 10
6.3. Matriz de confusión y errores 11
6.4. Efecto de los hiperparámetros 13
6.5. Discusión 13

7. Conclusiones **14**

A. Reproducibilidad y resultados por clase **16**

1. Introducción

La clasificación automática de texto consiste en asignar una o varias categorías a un documento a partir de su contenido. Es una tarea central en recuperación de información, filtrado de correo, análisis de opinión y moderación de contenido. A diferencia de los datos tabulares o de imágenes, el texto es secuencial, de longitud variable y disperso: la mayoría de las palabras de un documento no aparecen en los demás, y el significado depende del contexto y del orden.

En este proyecto se aborda la clasificación de *20 Newsgroups*, una colección de mensajes de Usenet agrupados en 20 temas. Aunque el enunciado del curso enmarca el proyecto como un problema de clasificación con múltiples etiquetas, los conjuntos de datos propuestos asignan una única etiqueta por documento; por tanto, el problema que los datos permiten evaluar es de **clasificación multiclase**. Esta distinción se explicita para evitar ambigüedades en la interpretación de las métricas.

Objetivos

Objetivo general. Diseñar, entrenar y evaluar una red neuronal Perceptrón Multicapa para clasificar documentos de 20 Newsgroups en sus 20 categorías, usando Keras/TensorFlow, y analizar críticamente su desempeño.

Objetivos específicos.

- Caracterizar el conjunto de datos y construir un preprocesamiento reproducible que evite fuga de información.
- Implementar una MLP con representación TF-IDF y justificar sus decisiones de arquitectura y regularización.
- Ajustar hiperparámetros mediante un protocolo experimental controlado y comparar al menos ocho configuraciones.
- Evaluar el modelo con exactitud, F1 macro, F1 ponderado, exactitud top-3, matriz de confusión y análisis por clase.
- Identificar los principales errores y limitaciones, y relacionarlos con las arquitecturas secuenciales de la segunda entrega.

Organización del documento

La Sección 2 resume los conceptos de TF-IDF y MLP. La Sección 3 describe el corpus y el preprocesamiento. La Sección 4 presenta la arquitectura. La Sección 5 detalla las condiciones de entrenamiento y el ajuste de hiperparámetros. La Sección 6 analiza los resultados y la Sección 7 concluye. El Apéndice A contiene los detalles de reproducibilidad y la tabla por clase.

2. Marco conceptual

2.1 Representación TF-IDF

Cada documento se representa como un vector disperso de pesos sobre un vocabulario V. Sea $tf(t, d)$ la frecuencia del término t en el documento d y $df(t)$ el número de documentos que contienen t. La

ponderación TF-IDF utilizada es

$$\text{tfidf}(t,d) = tf(t,d) \cdot \left(\ln \frac{1+N}{1+df(t)} + 1 \right), \tag{1}$$

donde N es el número de documentos de entrenamiento. Keras normaliza además cada vector a norma euclidiana unitaria. Esta representación atenúa las palabras muy frecuentes (“the”, “a”) y resalta las discriminativas, y ha sido durante décadas una línea base fuerte en clasificación de texto [2].

2.2 Perceptrón Multicapa

Una MLP es una red *feed-forward* formada por capas densas. Para una entrada $\mathbf{x} \in \mathbb{R}^d$, la capa l calcula

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right), \tag{2}$$

con $\sigma(z) = \max(0, z)$ (ReLU) en las capas ocultas. La capa de salida aplica softmax para obtener una distribución sobre las 20 clases,

$$p(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^{20} e^{z_j}}, \tag{3}$$

y se minimiza la entropía cruzada categórica

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^{B} \ln p(y_i \mid \mathbf{x}_i). \tag{4}$$

El entrenamiento usa retropropagación [3] y el optimizador Adam [4]. Para reducir el sobreajuste se emplea dropout [5], que desactiva aleatoriamente una fracción de las activaciones durante el entrenamiento, y parada temprana sobre la pérdida o exactitud de validación.

3. Conjunto de datos

3.1 Descripción

20 Newsgroups [1] reúne mensajes publicados en 20 grupos de noticias de Usenet en 1993. Los grupos cubren temas de computación, deportes, ciencia, religión y política. La colección original contiene alrededor de 20.000 documentos; la partición estándar de scikit-learn [6] asigna 11,314 documentos a entrenamiento y 7,532 a prueba, con 20 clases.

Tabla 1: Estadísticas descriptivas del corpus tras eliminar cabeceras, pies de página y citas.

Propiedad	Valor
Documentos de entrenamiento	11,314
Documentos de prueba	7,532
Clases	20
Palabras por documento (media)	185.8
Palabras por documento (mediana)	83
Percentil 95 de longitud	572
Longitud máxima (palabras)	11,765
Documentos sin palabras tras el filtrado	300
Razón de desbalance (clase mayor / menor)	1.59

El corpus está moderadamente desbalanceado: la clase más frecuente es `rec.sport.hockey` (600 documentos) y la menos frecuente `talk.religion.misc` (377 documentos), para una razón de 1.59× (Figura 1). Esta diferencia justifica reportar F1 macro además de la exactitud, pues una métrica global puede ocultar el mal desempeño en clases minoritarias.

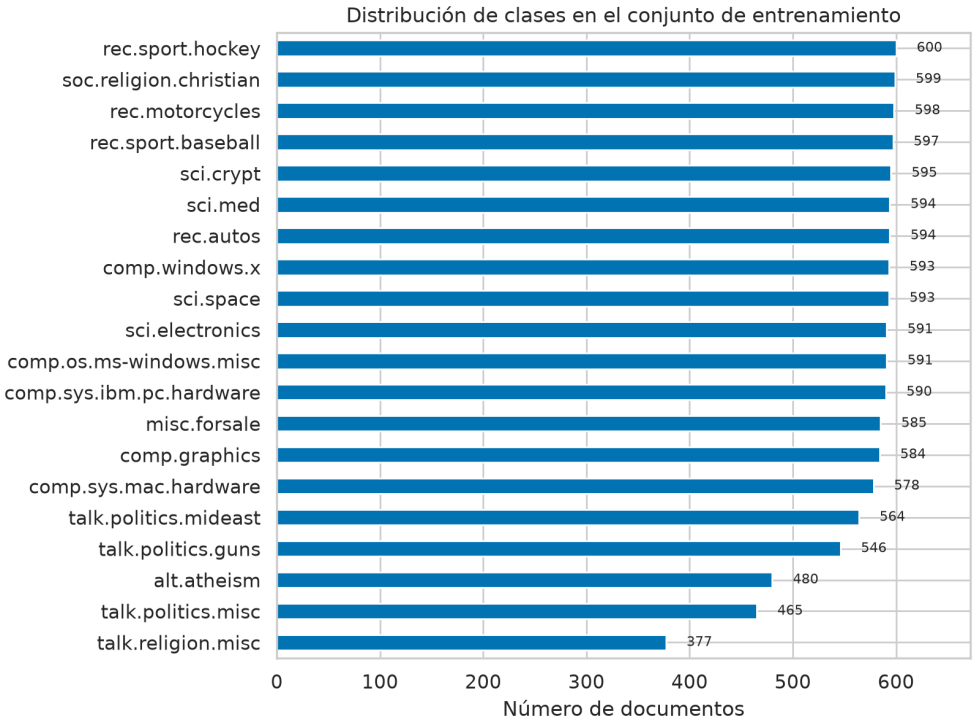


Figura 1: Distribución de las 20 clases en el conjunto de entrenamiento.

3.2 Preprocesamiento

El preprocesamiento se implementa en `src/data.py` y comprende:

1. **Eliminación de metadatos.** Se descartan cabeceras, pies de página y citas con la opción `remove=('headers','footers','quotes')` de scikit-learn. Sin este paso, campos como `Newsgroups:` o `Path:` contienen la etiqueta y producen una fuga de información severa.

2. **Texto crudo como entrada.** No se eliminan *stopwords* ni se aplica stemming; la capa `TextVectorization` pasa el texto a minúsculas, elimina puntuación y tokeniza por espacios.

3. **Vocabulario aprendido solo en entrenamiento.** El vectorizador se adapta con el conjunto de entrenamiento; validación y prueba se transforman con ese vocabulario. Esto evita que las estadísticas del test influyan en el modelo.

4. **Partición estratificada.** El 10 % del entrenamiento oficial se reserva para validación (1,132 documentos) y el 90 % restante (10,182 documentos) se usa para ajustar los pesos. El conjunto de prueba oficial (7,532 documentos) se utiliza una sola vez, al final.

La Figura 2 muestra que la mayoría de los documentos son cortos (mediana de 83 palabras), pero existen colas largas de hasta 11,765 palabras. Como TF-IDF no impone una longitud fija, los documentos largos no se truncan; esto es una diferencia importante frente a las RNN de la segunda entrega, que requieren secuencias de longitud acotada.

Un hallazgo del preprocesamiento es que 300 documentos del conjunto de entrenamiento quedan sin palabras después de eliminar cabeceras, pies y citas. Corresponden a mensajes cuyo contenido era únicamente metadatos o texto citado. Estos documentos se conservan para no alterar la partición oficial, pero son intrínsecamente inclasificables: el modelo recibe un vector TF-IDF nulo y predice la clase prioritaria. En el conjunto de prueba representan una fracción similar y establecen un techo práctico a la exactitud alcanzable.

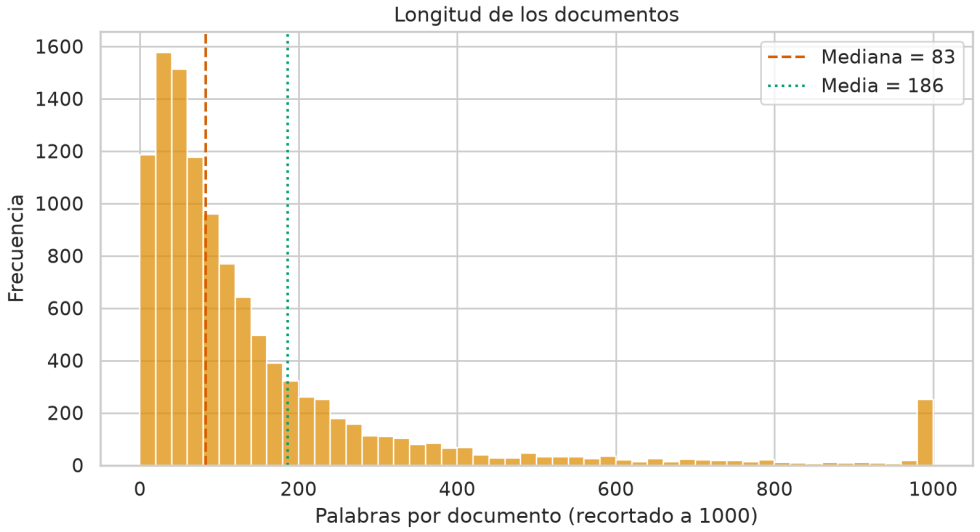


Figura 2: Distribución de la longitud de los documentos (recortada a 1000 palabras para visualización).

3.3 Análisis exploratorio

Además de la distribución de clases, se observó que muchos mensajes son respuestas dentro de un hilo y comparten vocabulario con otros grupos. Por ejemplo, mensajes de `comp.os.ms-windows.misc` discuten hardware y mensajes de `talk.politics.misc` discuten religión. Esta cercanía semántica anticipa las confusiones que se analizan en la Sección 6.

4. Arquitectura de la red

La arquitectura es una MLP que recibe texto y produce una distribución sobre 20 clases. La Figura 3 resume el flujo y la Tabla 2 detalla las dimensiones y el número de parámetros.

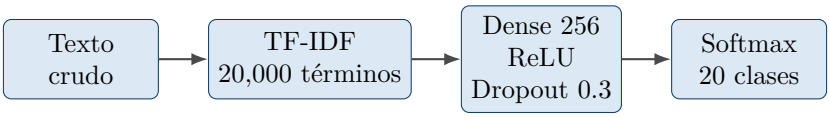


Figura 3: Arquitectura del Perceptrón Multicapa. La capa `TextVectorization` no tiene parámetros entrenables; aprende el vocabulario y calcula TF-IDF.

Tabla 2: Resumen de capas y parámetros del modelo principal.

Capa	Salida	Activación	Parámetros
`TextVectorization` (TF-IDF)	20.000	—	0
`Dense 1`	256	ReLU	5.120.256
`Dropout 1`	256	—	0
`Dense de salida`	20	Softmax	5.140
Total			**5,125,396**

4.1 Decisiones de diseño

Entrada TF-IDF en lugar de embeddings. Una alternativa habitual es aprender embeddings de palabras y promediarlos o concatenarlos. Sin embargo, para una MLP pura sobre un corpus pequeño, TF-IDF ofrece una señal densa y bien regularizada, entrena en segundos y sirve como línea base reproducible. Los embeddings sin contexto pierden gran parte de la información y requieren muchos más datos para ajustarse; se explorarán en la Entrega 2 con arquitecturas secuenciales.

Una capa oculta de 256 neuronas. Esta capa aprende combinaciones lineales de términos y es suficiente para un problema con 20 clases y 10,182 ejemplos. El barrido de hiperparámetros (Sección 6) mostró que añadir una segunda capa (512–256) reduce el F1 macro de 0.6663 a 0.6578 y que una tercera capa lo reduce a 0.6433; es decir, la capacidad adicional se dedica a memorizar el entrenamiento y no a generalizar. Se eligió por tanto la configuración más simple que obtuvo el mejor resultado.

Dropout. Se aplica una tasa de 0.3 después de la capa oculta, que concentra 5.1 millones de parámetros y por tanto el mayor riesgo de sobreajuste. La comparación con una tasa mayor (0.5–0.4) y con regularización L2 se presenta en la Sección 6.

Salida softmax y pérdida. Al ser clasificación multiclase mutuamente excluyente, se usa una neurona por clase con softmax y entropía cruzada categórica dispersa (`sparse_categorical_crossentropy`), que evita convertir las etiquetas a vectores one-hot.

5. Entrenamiento y ajuste de parámetros

5.1 Protocolo

Todos los experimentos comparten la misma partición, semilla (42), vocabulario adaptado solo en entrenamiento y criterio de parada temprana, de modo que las diferencias se puedan atribuir a la configuración evaluada. La Tabla 3 resume las condiciones.

Tabla 3: Condiciones de entrenamiento del modelo principal.

Elemento	Valor
Entrada	Texto crudo (sin cabeceras, pies ni citas)
Representación	TF-IDF, vocabulario de 20,000 términos
Optimizador	Adam, $\eta = 10^{-3}$
Pérdida	Entropía cruzada categórica dispersa
Batch	128
Épocas máximas	25
Parada temprana	`val_accuracy`, paciencia 5, $\Delta_{\text{mín}} = 10^{-4}$
Regularización	Dropout 0.3
Semilla	42 (determinismo activado)
Software	TensorFlow 2.20, Keras 3.15, scikit-learn 1.7, Python 3.12
Hardware	CPU Intel Core i5-9400F, 6 núcleos, 31 GB RAM

Se eligió la exactitud de validación como criterio de parada en lugar de la pérdida porque, en experimentos preliminares, la pérdida de validación aumentaba por sobreconfianza mientras la exactitud seguía mejorando. La parada temprana restaura los pesos de la mejor época, y el conjunto de prueba se evalúa una única vez con esos pesos.

5.2 Espacio de búsqueda

Se evaluaron 9 configuraciones:

- **Profundidad:** modelo lineal sin capas ocultas (regresión softmax sobre TF-IDF), una capa de 256 (principal), dos capas de 512–256 y tres capas de 512–256–128.
- **Regularización:** dropout 0.3 (principal), dropout 0.5–0.4 y L2 de 10^{-4} sobre los kernels.
- **Vocabulario:** 20,000 y 10.000 términos.
- **Balanceo:** pesos de clase inversamente proporcionales a su frecuencia.
- **Tasa de aprendizaje:** 10^{-3} y 5×10^{-4}.

La Tabla 4 presenta los resultados en el conjunto de prueba, ordenados por F1 macro. La Figura 4 los compara visualmente.

Tabla 4: Resultados de las configuraciones evaluadas en el conjunto de prueba, ordenadas por F1 macro.

Configuración	Capas	Drop.	Vocab.	Bal.	Parám.	Ép.	Exact.	F1	Top-3
MLP 1×256 (principal)	256	0.3	20k	No	5.1M	8	0.6771	0.6663	0.8452
MLP 2×512–256, lr 5e-4	512-256	0.4-0.3	20k	No	10.4M	10	0.6741	0.6653	0.8376
MLP 2×512–256	512-256	0.4-0.3	20k	No	10.4M	10	0.6680	0.6578	0.8240
MLP 2×512–256, balanceado	512-256	0.4-0.3	20k	Sí	10.4M	8	0.6686	0.6568	0.8258
MLP 2×512–256, dropout 0.5–0.4	512-256	0.5-0.4	20k	No	10.4M	9	0.6644	0.6566	0.8266
Lineal (sin capas ocultas)	—	-	20k	No	0.4M	11	0.6572	0.6451	0.8368
MLP 3×512–256–128	512-256-128	0.4-0.3-0.2	20k	No	10.4M	13	0.6498	0.6433	0.8140
MLP 2×512–256, L2 1e-4	512-256	0.4-0.3	20k	No	10.4M	12	0.6455	0.6393	0.8177
MLP 2×512–256, vocab. 10k	512-256	0.4-0.3	10k	No	5.3M	11	0.6439	0.6358	0.8123

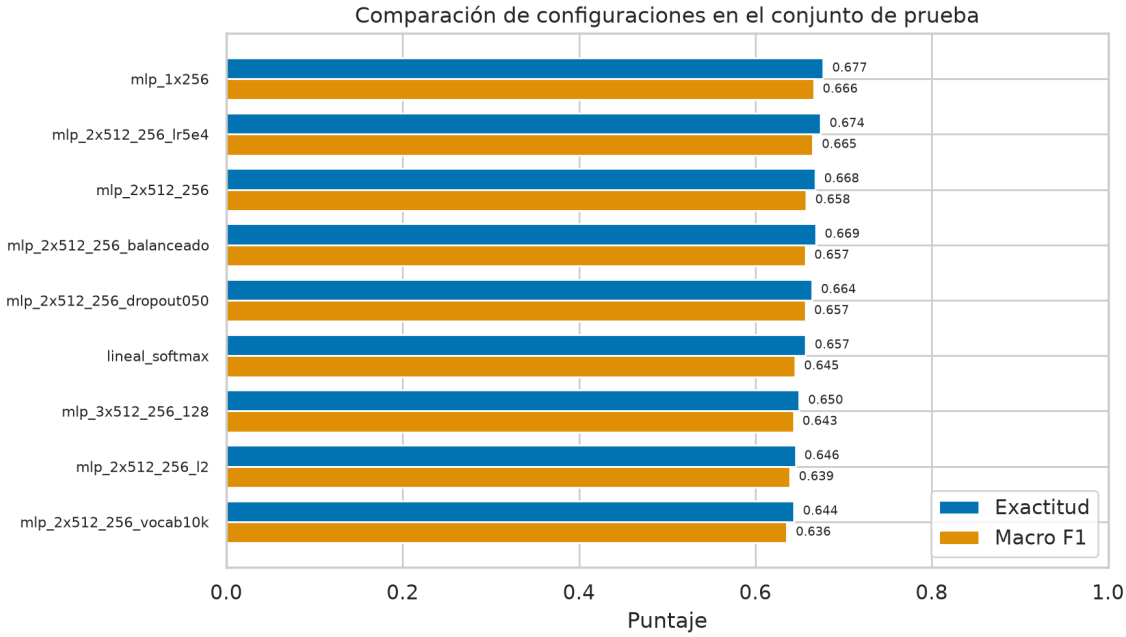


Figura 4: Exactitud y F1 macro en prueba para las configuraciones evaluadas.

5.3 Costo computacional

El modelo principal tiene 5.13 millones de parámetros y se entrenó en alrededor de 30 segundos en CPU (sin GPU), incluyendo la adaptación del vocabulario. La mayor parte del costo corresponde a la capa densa, que multiplica un vector TF-IDF de 20,000 dimensiones por una matriz de 20.000×256. Este costo es aceptable para un corpus de este tamaño y permite iterar rápidamente sobre hiperparámetros.

6. Resultados y análisis

6.1 Desempeño del modelo principal

La Tabla 5 reporta las métricas del modelo principal en el conjunto de prueba. El modelo se entrenó durante 8 épocas y la mejor época fue la 3, con una exactitud de validación de 0.7491.

Tabla 5: Métricas del modelo principal en el conjunto de prueba (7,532 documentos).

Métrica	Valor
Exactitud	0.6771
Precisión macro	0.6786
Recall macro	0.6655
F1 macro	0.6663
F1 ponderado	0.6762
Exactitud top-3	0.8452

La exactitud de 0.6771 es muy superior al azar (5 % para 20 clases balanceadas) y al *baseline* lineal, que obtiene 0.6572. La exactitud top-3 de 0.8452 indica que, en más de tres de cada cuatro documentos, la clase correcta está entre las tres más probables; esto es relevante en aplicaciones donde un humano revisa las sugerencias del modelo.

La brecha entre la exactitud de validación (0.7491) y la de prueba (0.6771) es de 7.20 puntos porcentuales. Parte de esta diferencia se explica por el tamaño reducido de la validación (1,132 documentos), que produce una estimación con alta varianza, y parte por el sobreajuste del modelo a los documentos de entrenamiento. Además, la distribución de longitudes y de grupos no es idéntica entre las particiones: el conjunto de prueba contiene mensajes de fechas y hilos distintos. La Figura 5 muestra la evolución típica: la exactitud de entrenamiento supera el 95 %, mientras la de validación se estabiliza entre el 72 % y el 75 %.

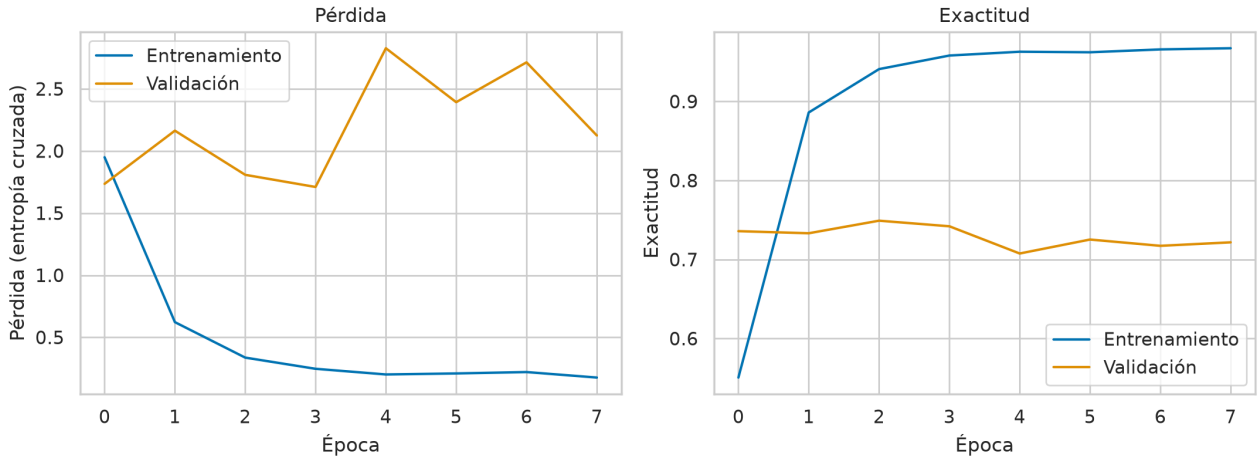


Figura 5: Curvas de pérdida y exactitud del modelo principal. La parada temprana restaura los pesos de la mejor época de validación.

6.2 Análisis por clase

La Figura 6 y la Tabla 7 (Apéndice A) muestran un desempeño heterogéneo. Las clases con mayor F1 son `rec.sport.hockey`, `comp.windows.x` y `misc.forsale`; las más difíciles son `talk.religion.misc`, `alt.atheism` y `talk.politics.misc`. Estas últimas son también las de menor soporte o las que comparten vocabulario con varias clases.

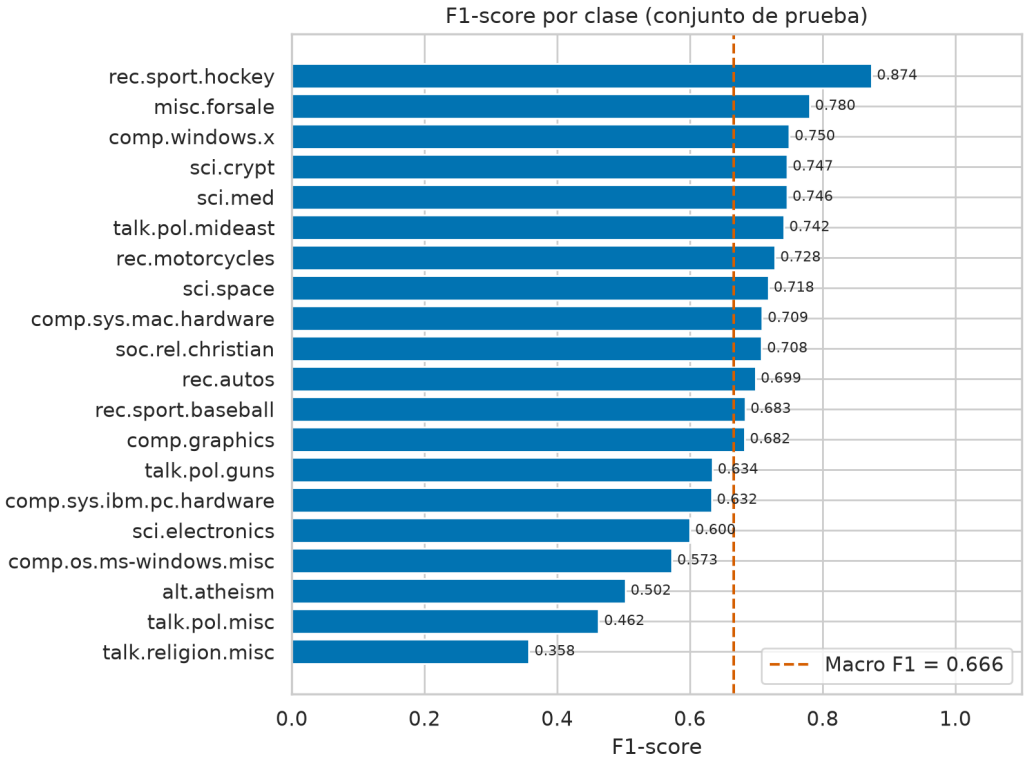


Figura 6: F1-score por clase en el conjunto de prueba.

6.3 Matriz de confusión y errores

La Figura 7 presenta la matriz de confusión normalizada por clase real. La diagonal concentra la mayor parte de la masa, pero se observan bloques fuera de ella:

- **Computación:** `comp.os.ms-windows.misc`, `comp.sys.ibm.pc.hardware` y `comp.sys.mac.hardware` se confunden entre sí porque comparten términos de hardware, sistemas operativos y controladores.
- **Religión:** `alt.atheism`, `soc.religion.christian` y `talk.religion.misc` comparten vocabulario y argumentos; además, los hilos de discusión mezclan las tres perspectivas.
- **Política:** `talk.politics.guns`, `talk.politics.mideast` y `talk.politics.misc` presentan solapamiento temático y de entidades nombradas.

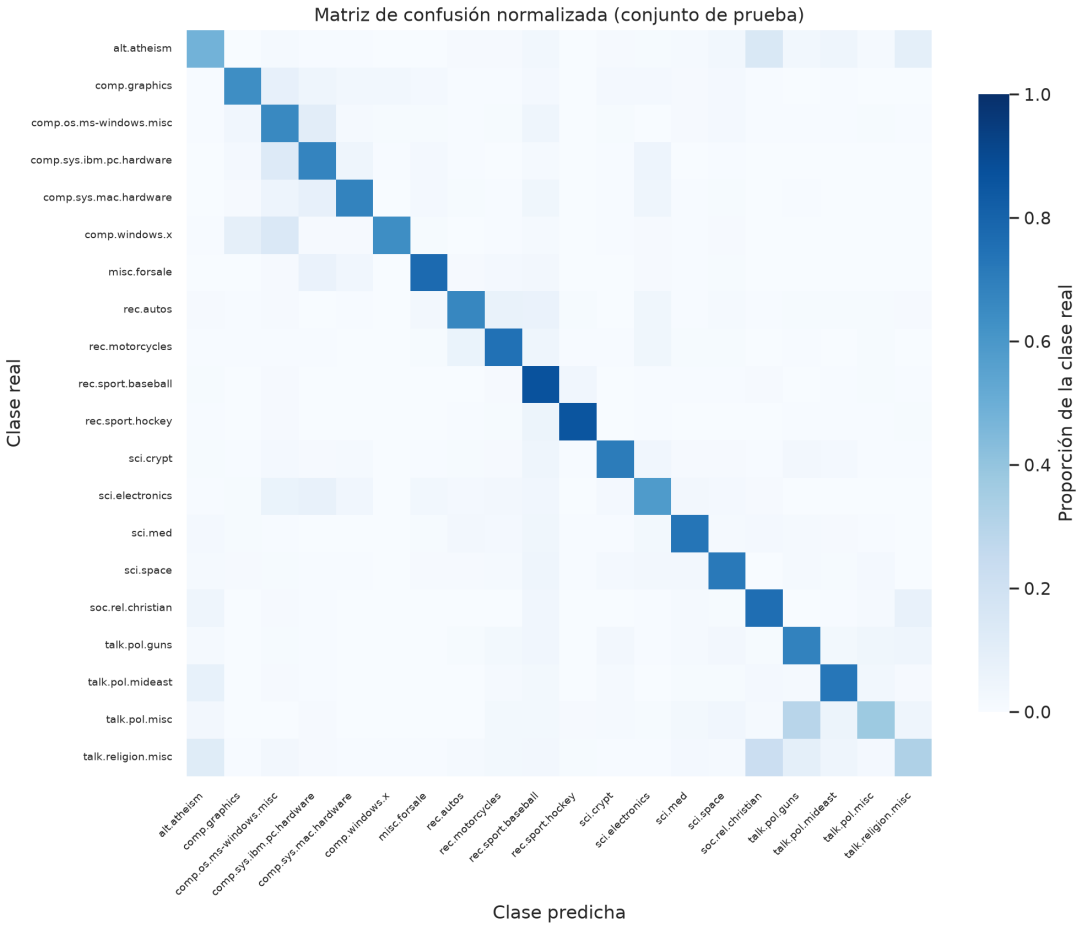


Figura 7: Matriz de confusión normalizada por clase real del modelo principal.

Tabla 6: Confusiones más frecuentes del modelo principal.

Clase real	Clase predicha	Casos	% de la clase real
talk.politics.misc	talk.politics.guns	91	29.4 %
comp.windows.x	comp.os.ms-windows.misc	58	14.7 %
talk.religion.misc	soc.religion.christian	56	22.3 %
comp.sys.ibm.pc.hardware	comp.os.ms-windows.misc	52	13.3 %
alt.atheism	soc.religion.christian	48	15.0 %
comp.os.ms-windows.misc	comp.sys.ibm.pc.hardware	42	10.7 %
comp.windows.x	comp.graphics	36	9.1 %
comp.graphics	comp.os.ms-windows.misc	33	8.5 %

Un análisis cualitativo de los errores muestra que muchos documentos mal clasificados son cortos, con pocas palabras discriminativas, o son respuestas que citan el mensaje anterior y comparten su vocabulario. TF-IDF no distingue el tema principal del documento de las palabras citadas, lo que sugiere que un modelo secuencial podría mejorar al modelar el orden y el contexto.

6.4 Efecto de los hiperparámetros

Profundidad. El modelo lineal sin capas ocultas obtiene 0.6572 de exactitud y 0.6451 de F1 macro. Una capa de 256 neuronas (modelo principal) mejora a 0.6771 y 0.6663. Añadir una segunda capa (512–256) baja a 0.6680 y 0.6578, y una tercera capa (512–256–128) baja a 0.6498 y 0.6433. La ganancia proviene de la primera capa oculta; la capacidad adicional de las redes más profundas se dedica a memorizar el entrenamiento y no aporta información nueva.

Dropout. Aumentar el dropout a 0.5–0.4 en la red de dos capas produce 0.6644 de exactitud y 0.6566 de F1 macro, por debajo del modelo principal. Un dropout excesivo en la primera capa dificulta el aprendizaje de las combinaciones de términos, mientras que una tasa menor aumenta el sobreajuste; 0.3 resulta un compromiso razonable para una sola capa de 256 neuronas.

Vocabulario. Reducir el vocabulario a 10.000 términos reduce la exactitud a 0.6439 y el F1 macro a 0.6358. El corpus contiene muchos términos técnicos y nombres propios poco frecuentes que son informativos; truncar el vocabulario los convierte en `[UNK]` y elimina señal.

Balanceo de clases. Los pesos balanceados producen 0.6686 de exactitud y 0.6568 de F1 macro. El F1 macro no supera al del modelo sin balanceo, lo que sugiere que el desbalance del corpus no es lo bastante severo para justificar el costo de reponderar las clases.

Tasa de aprendizaje y L2. La tasa 5×10^{-4} obtiene 0.6741 y 0.6653, y L2 10^{-4} obtiene 0.6455 y 0.6393. Ninguna supera al modelo principal: una tasa menor requiere más épocas y L2 compite con el dropout sin aportar una mejora clara.

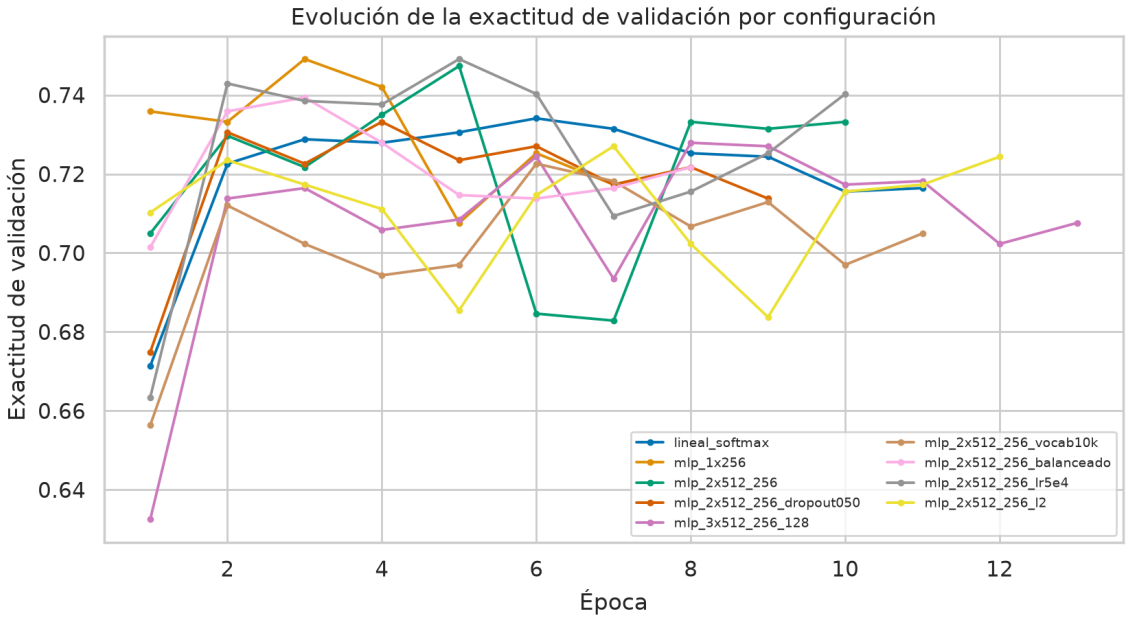


Figura 8: Exactitud de validación por época para las configuraciones evaluadas.

6.5 Discusión

Los resultados son coherentes con la literatura: los modelos lineales sobre TF-IDF son líneas base fuertes en 20 Newsgroups, y las redes neuronales aportan mejoras moderadas cuando se controla el sobreajuste [2, 1]. La principal limitación de la arquitectura es que la representación *bag-of-words* ignora el orden de las palabras y la estructura del discurso. Además, el vocabulario se aprende solo

del entrenamiento y no incorpora conocimiento semántico externo (por ejemplo, embeddings preentrenados). A esto se suman los 300 documentos de entrenamiento que quedan vacíos tras eliminar metadatos y citas: son inclasificables y el modelo solo puede aprender de ellos la distribución prioritaria, lo que introduce ruido y reduce la exactitud máxima alcanzable.

Otra limitación metodológica es que la selección de hiperparámetros se hizo con un único split de validación de 1,132 documentos y una sola semilla. Las diferencias de uno o dos puntos porcentuales entre configuraciones pueden estar dentro del ruido de estimación. Para la entrega final sería recomendable usar validación cruzada o varias semillas y reportar media y desviación estándar. También conviene repetir la evaluación con la métrica F1 macro como criterio principal de selección, tal como se hizo en este trabajo.

Finalmente, el desbalance de clases y las confusiones entre grupos temáticamente cercanos sugieren que parte del error es irreducible con bolsas de palabras: dos documentos pueden compartir casi todo el vocabulario y pertenecer a grupos distintos porque el grupo de noticias es una convención social, no un tema puro.

7. Conclusiones

Se construyó y evaluó una MLP en Keras/TensorFlow para clasificar 20 Newsgroups en 20 categorías. El modelo combina una representación TF-IDF de 20,000 términos con una capa densa de 256 neuronas, dropout 0.3 y salida softmax. En el conjunto de prueba obtuvo una exactitud de 0.6771, un F1 macro de 0.6663 y una exactitud top-3 de 0.8452, superando al azar y a un modelo lineal.

El ajuste de hiperparámetros mostró que una sola capa oculta es suficiente, que añadir capas reduce el F1 macro, que un vocabulario más pequeño pierde señal y que el balanceo de clases no compensa la pérdida de exactitud. El dropout 0.3 y el vocabulario completo ofrecen el mejor compromiso. La brecha entre validación y prueba y el análisis de confusiones evidencian sobreajuste y solapamiento semántico entre clases.

Como trabajo futuro para la Entrega 2, se propone reemplazar la representación TF-IDF por secuencias de tokens y comparar una RNN sin memoria (por ejemplo, una red densa sobre embeddings o una RNN simple) con una RNN con memoria (LSTM/GRU) o un Transformer. Estas arquitecturas pueden modelar el orden, el contexto y las dependencias de largo plazo que TF-IDF ignora, y se espera que mejoren especialmente en las clases con mayor confusión.

Referencias

[1] K. Lang, “NewsWeeder: Learning to Filter Netnews,” in *Proceedings of the 12th International Conference on Machine Learning (ICML)*, 1995, pp. 331–339.

[2] T. Joachims, “Text Categorization with Support Vector Machines: Learning with Many Relevant Features,” in *Proceedings of the 10th European Conference on Machine Learning (ECML)*, 1998, pp. 137–142.

[3] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning Representations by Back-Propagating Errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[4] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.

[5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] M. Abadi *et al.*, "TensorFlow: A System for Large-Scale Machine Learning," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.

A. Reproducibilidad y resultados por clase

Estructura del código

```
Proyecto/
|-- src/
|   |-- config.py       # hiperparámetros y rutas
|   |-- data.py         # descarga, cacheo y partición
|   |-- model.py        # vectorizador TF-IDF y MLP
|   |-- train.py        # entrenamiento y evaluación
|   |-- experiments.py  # barrido de configuraciones
|   |-- plots.py        # figuras
|   |-- predict.py      # demo por línea de comandos
|   `-- build_docs.py   # tablas y macros del informe
|-- notebooks/
|   `-- entrega1_mlp.ipynb
|-- results/            # métricas, modelos y figuras
|-- informe/            # este documento
`-- presentacion/       # diapositivas
```

Comandos

```
uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python -r requirements.txt

# Entrenar las 9 configuraciones y generar resultados
.venv/bin/python -m src.experiments

# Predicción interactiva
.venv/bin/python -m src.predict "texto a clasificar"

# Regenerar tablas y compilar el informe (requiere Tectonic instalado)
.venv/bin/python -m src.build_docs
tectonic -X compile informe/informe.tex
```

Resultados por clase

Tabla 7: Métricas por clase del modelo principal en el conjunto de prueba.

Clase	Precisión	Recall	F1	Soporte
alt.atheism	0.524	0.483	0.502	319
comp.graphics	0.730	0.640	0.682	389
comp.os.ms-windows.misc	0.506	0.660	0.573	394
comp.sys.ibm.pc.hardware	0.594	0.676	0.632	392
comp.sys.mac.hardware	0.740	0.681	0.709	385
comp.windows.x	0.904	0.641	0.750	395
misc.forsale	0.786	0.774	0.780	390
rec.autos	0.737	0.664	0.699	396
rec.motorcycles	0.711	0.746	0.728	398
rec.sport.baseball	0.563	0.869	0.683	397
rec.sport.hockey	0.891	0.857	0.874	399
sci.crypt	0.791	0.707	0.747	396
sci.electronics	0.615	0.585	0.600	393
sci.med	0.761	0.732	0.746	396
sci.space	0.715	0.721	0.718	394
soc.religion.christian	0.664	0.759	0.708	398
talk.politics.guns	0.588	0.687	0.634	364
talk.politics.mideast	0.755	0.729	0.742	376
talk.politics.misc	0.597	0.377	0.462	310
talk.religion.misc	0.401	0.323	0.358	251
Macro promedio	0.679	0.666	0.666	7,532